3D Pole Balancing for Spiking NEAT

A first standalone benchmark for Rhayader's spiking NEAT: a cart moving freely on a plane, balancing a rigid pole that can tip in any direction. Orientation is held as a unit quaternion, the dynamics come from a Lagrangian, and time stepping uses a Lie-group integrator. The aim is a demo that is physically correct, holds up to scrutiny, and shows the maths behind it.

Why this task

- It tests spiking NEAT on its own before SDR wiring is added, so failures can be traced to the evolution rather than the input plumbing.
- The classic 2D cart-pole is a well-known NEAT benchmark. Taking it to 3D with quaternion dynamics is a clear step beyond the standard.
- It extends naturally to swing-up and to the double pole later, and quaternions handle both without singularities.

Physical model

Bodies

- Cart of mass M at position r = (x, y) on a horizontal plane, driven by a horizontal force F = (Fx, Fy).
- Thin rigid pole of mass m and half-length l (pivot to centre of mass), attached to the cart by a frictionless ball joint.

Orientation

- Unit quaternion q maps the pole's body frame to the world frame. The pole's long axis is the body z-axis, so the pole's direction in the world is d = q · e3 · q*.
- Body angular velocity Ω = (Ω1, Ω2, Ω3). Kinematics: $\dot{q} = \tfrac{1}{2}\, q \otimes (0, \Omega)$.
- A thin rod has essentially no inertia about its own axis, so spin about that axis (Ω3) does nothing. Either fix Ω3 = 0 and work with (Ω1, Ω2), or give the rod a small axial inertia so the inertia matrix is invertible. The first option is cleaner. The second is simpler to code.

Lagrangian

The pole's centre of mass is at p = r + l d. With J the pole's inertia about its centre of mass (transverse moment m $l^2/3$ for a uniform rod of length 2l):

$$L = \tfrac{1}{2} M |\dot{r}|^2 + \tfrac{1}{2} m |\dot{p}|^2 + \tfrac{1}{2} \Omega^T J \Omega - m g l (e3 \cdot d)$$

Here $\dot{p} = \dot{r} + l \cdot R(\Omega \times e3)$, and R is the rotation matrix of q. The applied force F acts on the cart as an external, non-conservative generalised force. Any friction is added the same way.

Equations of motion

Since the orientation lives on the rotation group SO(3) rather than in flat coordinates, the rotational equations are the forced Euler–Poincaré equations rather than the ordinary Euler–Lagrange ones:

- Translational: $d/dt(\partial L/\partial \dot{r}) - \partial L/\partial r = F$
- Rotational: $d/dt(\partial L/\partial \Omega) + \Omega \times \partial L/\partial \Omega = M_g$, where M_g is the moment from gravity and from the coupling to the cart, written in the body frame.

The term $\Omega \times \partial L/\partial \Omega$ is what the group structure adds. It is the rigid-body equivalent of the terms people get wrong when they derive cart-pole equations by hand.

Integration

Tier 1, explicit Lie-group stepping (get it running first)

- Solve the equations of motion for accelerations ($\ddot{r}$, $\dot{\Omega}$) each step, using a mass matrix and a force vector.
- Update velocities, then update orientation by rotation rather than addition: $q \leftarrow q \otimes \exp(\tfrac{1}{2} h \Omega)$, where exp is the quaternion exponential of a pure-vector quaternion.
- Because the update composes rotations, q stays unit length up to floating-point error. Renormalise occasionally as a tidy-up, not as a correction.
- Time step $h = 0.02$ s, matching the classic benchmark.

Tier 2, Lie-group variational integrator (the impressive version)

- Discretise the action rather than the equations: the discrete Lagrangian $L_d(q_k, q_{k+1})$ approximates the integral of L over one step.
- The discrete Euler–Poincaré equations give an implicit equation for the relative rotation $F_k = q_k^* \otimes q_{k+1}$, solved by a few Newton iterations.
- This version is symplectic and momentum-preserving by construction, so energy shows no long-term drift with $F = 0$ and no friction.
- Reference: T. Lee, M. Leok and N. H. McClamroch on Lie-group variational integrators for rigid bodies and the 3D pendulum.

Haskell implementation notes

- **ad package.** Write the kinetic and potential energy as ordinary polymorphic functions. Use grad and hessian to get the mass matrix (the Hessian of kinetic energy with respect to the velocities) and the generalised forces. For Tier 2, use jacobian for the Newton steps.
- **linear package.** Linear.Quaternion provides quaternion multiplication, rotate, and a Floating instance with exp. Keep the energy functions polymorphic in the number type so ad can differentiate through them.
- **Separate state.** Keep a record of cart position, cart velocity, orientation and angular velocity. The integrator is a pure function from (state, force) to the next

state, which fits the existing MonadClock / SimClock split.
- **Visualisation.** raylib uses quaternions for rotation, so the orientation goes straight into the existing h-raylib visualiser.

Interface to the spiking network

Inputs (via SDR scalar encoders)

- Cart position (x, y) and velocity (ẋ, ẏ): four values.
- Pole tip direction d as a 3D unit vector, rather than raw quaternion components. It is smooth, has no double cover (q and −q give the same rotation), and encodes naturally.
- Pole angular velocity in the transverse plane: two values.
- Each value goes through a scalar SDR encoder, and active bits inject spikes at the start of each control window.

Outputs

- Four output neurons for +x, −x, +y and −y, with lateral inhibition within each axis pair.
- First version: bang-bang control on each axis, using the first spike in the window. Later: graded force from spike timing or count.

Episode and fitness

- Failure when the pole's tilt from vertical exceeds a threshold (the 2D classic uses 12°; start wider, say 30°, while evolution gets going) or when the cart leaves a circular arena.
- Fitness is steps balanced, up to a cap. Optionally add a penalty for jitter so evolution doesn't settle on frantic oscillation.

Starting parameters (classic values, metric)

- g = 9.81 m/s², M = 1.0 kg, m = 0.1 kg, l = 0.5 m, maximum force 10 N per axis, h = 0.02 s, arena radius 2.4 m.

SDR encoders and decoders

Each encoder is a new instance of the existing encoder typeclass in Data.SDR. The encoded pieces are concatenated into one input SDR.

Encoder	What it does	Use in the pole task
Scalar	A contiguous run of active bits; nearby values share bits	Quick first version for cart position and velocity
Random distributed scalar (RDSE)	Hashes value buckets to random bits; nearby buckets overlap; no fixed range	Robust replacement for the scalar encoder on all continuous inputs

Encoder	What it does	Use in the pole task
Cyclic	Scalar encoder whose range wraps round	Angles, if any are encoded directly; time of day in later tasks
Category	Unrelated bits per category	Discrete modes, such as which player in the jousting game
Coordinate / grid-cell	Encodes a position in 2D or 3D so nearby points overlap, at several scales	Cart position on the plane; pole tip direction as a point on the unit sphere

Design checks for every encoder

- Similar inputs give overlapping SDRs, and overlap falls off smoothly with distance.
- Sparsity stays fixed (for example around 2% active bits), whatever the value.
- Resolution is chosen to match the control problem. For instance, tilt needs fine resolution near upright, where control matters most.

Decoders (from spikes or SDRs back to actions or values)

- **First-spike with lateral inhibition.** The first output neuron to fire in a window wins. Fast and simple, and the planned starting point.
- **Spike count or timing to graded output.** Maps how early or how often an output fires to a force magnitude.
- **Learned classifier.** An HTM-style SDR classifier learns which output bits map to which values. Useful for prediction tasks and for reading internal columns.

Open question: how the decoder choice interacts with fitness, since bang-bang control evolves faster but graded control balances more smoothly.

Validation before evolving anything

1. **Planar reduction.** Constrain motion to the x–z plane and compare against the classic frictionless 2D equations (Barto, Sutton and Anderson 1983, with Florian's 2007 corrections). Trajectories should match closely for small time steps.
2. **Energy check.** With F = 0 and no friction, total energy should stay bounded in Tier 1 and show no secular drift in Tier 2.
3. **Quaternion norm.** Log $|q| - 1$ over long runs.
4. **Symmetry.** A pole released at a small tilt in any compass direction should fall identically, rotated. This catches axis and sign mistakes.
5. **Hand controller.** A simple PD controller on each axis should balance the pole. If it can't, the physics is wrong, not the network.

Roadmap

1. Physics core (Tier 1), planar validation and energy checks.

2. Visualiser hookup in h-raylib.
3. PD-controller sanity check.
4. Spiking NEAT on the 3D task with simple direct encoding, standalone.
5. SDR encoders on the inputs, first-spike decoding on the outputs.
6. Tier 2 variational integrator.
7. Extensions: swing-up from hanging, double pole, recorded runs for the crowdfunding video and the mathematical write-up.

Spin-off: pole jousting game

Two players each steer a cart with a joystick, keep their own pole balanced, and try to knock the other's pole over.

- **Input.** raylib's built-in gamepad support maps each analogue stick to the force on one cart.
- **New physics.** Each pole is modelled as a capsule (a cylinder with rounded ends). Detect capsule–capsule contact each step and apply an impulse at the contact point, which feeds into both poles' angular velocity and both carts. Cart–cart contact works the same way.
- **Modes.** Human vs human; human vs an evolved spiking network; network vs network.
- **Research tie-in.** Network-vs-network jousting is a coevolution arena in the spirit of the original Ruby NEAT critters, where the winner mates. It tests reactive control under adversarial disturbance, which standard pole balancing doesn't.

Research claim (to settle early)

The physics and the game show the craft. Academic standing comes from a new contribution that others have checked, usually a published paper. That needs one sharp, testable claim, decided before the experiments, for example:

- Evolved spiking columns reach balance with fewer neurons, fewer evaluations, or less energy than a standard baseline (rate-coded NEAT, or a small backprop-trained controller).
- Spiking controllers recover from adversarial knocks (the jousting setting) more robustly than the baselines.
- The same evolved controller transfers from 2D to 3D, or from single to double pole, with little retraining.

Open question: which claim to pursue, and which baselines to compare against.

Common pitfalls (lessons from the 2D version)

- Using the full pole length where the equations want the half-length.
- Computing cart acceleration before pole acceleration in the explicit 2D form.

- Inconsistent sign conventions for tilt and force.
- Using spherical coordinates for the pole, which are singular exactly at upright. Quaternions avoid this.